Guía para Actividad Práctica: App Móbil con Acceso a Base de Datos MySQL (React Native - Expo)

UTN FRH TUP Programación 3 — Desarrollo Mobile con Node.js en Windows, API REST (https://progra3/api/base/) y Tabla productos01

Prof. Osvaldo Cantone

En esta guía paso a paso construiremos una aplicación móvil completa en React Native utilizando Expo sobre un entorno Windows con Node.js instalado de forma nativa. La aplicación se conectará a través de HTTP/HTTPS al endpoint remoto https://progra3/api/base/ para consultar y presentar en la pantalla del teléfono los registros de la tabla productos01 (id, descripción, marca y stock).

1. Arquitectura de Comunicación

Por razones de seguridad y arquitectura, una aplicación móvil en React Native no realiza conexiones TCP directas al motor MySQL (puerto 3306). En su lugar, trabaja en un esquema de tres capas:

- **1. Capa de Presentación (App Expo):** Ejecuta la interfaz gráfica en el teléfono (Expo Go) y realiza peticiones HTTP asíncronas mediante fetch() hacia https://progra3/api/base/.
- **2. Capa de Backend / API (Node.js):** Recibe la petición en la ruta https://progra3/api/base/, se conecta internamente al motor MySQL, ejecuta la consulta SQL y devuelve los datos en formato JSON.
- **3. Capa de Datos (Servidor MySQL):** Almacena la base de datos 'datos' y la tabla 'productos01' con la información de inventario.

```
[ Teléfono (App Expo) ]  <--- HTTPS / JSON (https://progra3/api/base/) --->  [ Servidor API ]  <--- SQL --->  [ MySQL: productos01 ]
```

2. Paso 1: Despliegue de la Base de Datos y Tabla en MySQL

El primer paso consiste en preparar el esquema de datos en el servidor MySQL. Ejecuta el siguiente script SQL desde la consola de MySQL o desde phpMyAdmin para crear la base de datos, el usuario de conexión y la tabla productos01 con datos de prueba:

```
-- 1. Crear la base de datos (si no existe)
CREATE DATABASE IF NOT EXISTS datos CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

-- 2. Crear el usuario de aplicación y otorgar permisos
CREATE USER IF NOT EXISTS 'prog3'@'%' IDENTIFIED BY 'admin123';
GRANT ALL PRIVILEGES ON datos.* TO 'prog3'@'%';
```

```
FLUSH PRIVILEGES;

-- 3. Seleccionar la base de datos
USE datos;

-- 4. Crear la tabla productos01 (id, descripcion, marca, stock)
CREATE TABLE IF NOT EXISTS productos01 (
    id INT AUTO_INCREMENT PRIMARY KEY,
    descripcion VARCHAR(150) NOT NULL,
    marca VARCHAR(80) NOT NULL,
    stock INT NOT NULL DEFAULT 0
);

-- 5. Insertar registros iniciales de prueba
INSERT INTO productos01 (descripcion, marca, stock) VALUES
('Monitor LED 24 pulgadas Full HD', 'Samsung', 15),
('Teclado Mecánico RGB Switch Blue', 'Redragon', 8),
('Mouse Inalámbrico Ergonómico', 'Logitech', 25),
('Auriculares Bluetooth con Cancelación', 'Sony', 3),
('Disco Sólido SSD NVMe 1TB', 'Kingston', 0),
('Webcam Full HD 1080p con Micrófono', 'Logitech', 12);
```

3. Paso 2: Configuración del Endpoint en el Servidor (https://progra3/api/base/)

Para que la URL https://progra3/api/base/ responda a las peticiones de nuestra aplicación móvil devolviendo el contenido de productos01 en formato JSON, configuramos el servicio en el servidor. A continuación se incluyen las dos implementaciones posibles (Node.js con Express y PHP con PDO):

Opción A: Servicio Backend en Node.js (server.js)

En el directorio del backend del servidor, inicializamos el proyecto e instalamos las dependencias:

```
mkdir api-base
cd api-base
npm init -y
npm install express cors mysql2
```

Código del archivo server.js:

```
const express = require('express');
const mysql = require('mysql2/promise');
```

```
const cors = require('cors');

const app = express();
app.use(cors());
app.use(express.json());

// Pool de conexiones al motor MySQL
const pool = mysql.createPool({
  host: '127.0.0.1',
  user: 'prog3',
  password: 'admin123',
  database: 'datos',
  waitForConnections: true,
  connectionLimit: 10
});

// Endpoint expuesto en /api/base/
app.get('/api/base/', (req, res) => {
  pool.query('SELECT id, descripcion, marca, stock FROM productos01')
    .then(([rows]) => {
      res.json({
        status: 'success',
        data: rows
      });
    })
    .catch(err => {
      console.error('Error al consultar productos01:', err.message);
      res.status(500).json({
        status: 'error',
        message: 'Error al consultar la base de datos'
      });
    });
});

app.listen(3000, '0.0.0.0', () => {
  console.log('API escuchando en el puerto 3000 (ruta /api/base/)');
});
```

4. Paso 3: Verificación del Entorno Node.js en Windows

En nuestra computadora con sistema operativo Windows, trabajamos con Node.js instalado de forma nativa. Antes de crear el proyecto móvil, abrimos una terminal de PowerShell o CMD y verificamos que Node y npm respondan correctamente:

```
node -v
npm -v
npx -v
```

> 💡 **Recordatorio sobre el entorno en Windows**
>
> No es necesario instalar React ni Expo de forma global en Windows. Con tener Node.js LTS instalado, la herramienta npx descargará y configurará las versiones exactas dentro de la carpeta de nuestro proyecto.

5. Paso 4: Creación del Proyecto Expo con npx

Ubicados en nuestra carpeta de trabajo en Windows (por ejemplo, C:\localhost\P3.M32.26\mobile), ejecutamos el comando npx para crear el proyecto con la plantilla en blanco (blank) en JavaScript puro:

```
# 1. Posicionarse en la carpeta de proyectos móviles
cd C:\localhost\P3.M32.26\mobile

# 2. Crear el proyecto Expo con versión y plantilla definidas
npx create-expo-app@5.0.0 appProductos01 --template blank@57

# 3. Ingresar al directorio del proyecto creado
cd appProductos01

# 4. Abrir el proyecto en Visual Studio Code
code .
```

Detalle de los parámetros utilizados en el comando npx:

- • **npx create-expo-app@5.0.0:** Ejecuta el generador oficial de proyectos de Expo sin requerir instalación global previa.
- • **appProductos01:** Es el nombre de la carpeta y del proyecto que se creará.
- • **--template blank@57:** Indica que deseamos una estructura mínima en JavaScript (un único archivo App.js, sin TypeScript ni ruteo complejo) compatible con el SDK 57 de la app Expo Go del teléfono.

6. Paso 5: Código de la Aplicación Móvil (App.js)

Abrimos el archivo App.js en la raíz del proyecto en Visual Studio Code y reemplazamos todo su contenido por el siguiente código. La aplicación realiza la petición fetch() hacia https://progra3/api/base/ al iniciarse y presenta cada registro de la tabla productos01 en una tarjeta visual:

```
import React, { useState, useEffect } from 'react';
import {
  StyleSheet,
  Text,
  View,
  FlatList,
  ActivityIndicator,
  SafeAreaView,
  StatusBar,
  TouchableOpacity
} from 'react-native';

// URL del endpoint que consulta la base de datos
const API_URL = 'https://progra3/api/base/';

export default function App() {
  // Estados para almacenar la lista de productos, el estado de carga y errores
  const [productos, setProductos] = useState([]);
  const [cargando, setCargando] = useState(true);
  const [error, setError] = useState(null);

  // Función que consulta el endpoint remoto mediante Promesas (.then / .catch)
  const consultarProductos = () => {
    setCargando(true);
    setError(null);

    fetch(API_URL)
      .then(response => {
        if (!response.ok) {
          throw new Error(`Error HTTP: ${response.status}`);
        }
        return response.json();
      })
      .then(json => {
        if (json.status === 'success') {
          setProductos(json.data);
        } else if (Array.isArray(json)) {
```

```
        // Soporte directo por si el endpoint devuelve el array de filas directamente
        setProductos(json);
      } else {
        throw new Error(json.message || 'Formato de respuesta no reconocido');
      }
    })
    .catch(err => {
      console.error('Error al conectar con la base de datos:', err);
      setError('No se pudo obtener la información desde https://progra3/api/base/');
    })
    .finally(() => {
      setCargando(false);
    });
};

// useEffect ejecuta la consulta automáticamente al abrir la pantalla
useEffect(() => {
  consultarProductos();
}, []);

// Función encargada de dibujar cada fila de la tabla productos01
const renderProducto = ({ item }) => {
  const sinStock = Number(item.stock) === 0;

  return (
    <View style={styles.card}>
      <View style={styles.cardHeader}>
        <Text style={styles.idTexto}>ID #{item.id}</Text>
        <Text style={styles.marcaTexto}>{item.marca}</Text>
      </View>

      <Text style={styles.descripcionTexto}>{item.descripcion}</Text>

      <View style={styles.cardFooter}>
        <Text style={[
          styles.stockBadge,
          sinStock ? styles.sinStock : styles.conStock
        ]}>
          {sinStock ? 'SIN STOCK' : `Stock: ${item.stock} unidades`}
        </Text>
      </View>
    </View>
  );
```

```
  };

  return (
    <SafeAreaView style={styles.container}>
      <StatusBar barStyle="dark-content" backgroundColor="#f0f4f8" />

      <View style={styles.encabezado}>
        <Text style={styles.titulo}>Catálogo de Productos</Text>
        <Text style={styles.subtitulo}>Tabla: productos01 | Origen:
https://progra3/api/base/</Text>
      </View>

      {cargando ? (
        <View style={styles.centro}>
          <ActivityIndicator size="large" color="#2b6cb0" />
          <Text style={styles.textoEstado}>Consultando base de datos...</Text>
        </View>
      ) : error ? (
        <View style={styles.centro}>
          <Text style={styles.textoError}>{error}</Text>
          <TouchableOpacity style={styles.botonReintentar} onPress={consultarProductos}>
            <Text style={styles.textoBoton}>Reintentar consulta</Text>
          </TouchableOpacity>
        </View>
      ) : (
        <FlatList
          data={productos}
          keyExtractor={(item) => item.id.toString()}
          renderItem={renderProducto}
          contentContainerStyle={styles.lista}
          onRefresh={consultarProductos}
          refreshing={cargando}
          ListEmptyComponent={
            <Text style={styles.textoEstado}>No hay productos cargados en
productos01.</Text>
          }
        />
      )}
    </SafeAreaView>
  );
}

const styles = StyleSheet.create({
```

```
  container: {
    flex: 1,
    backgroundColor: '#f0f4f8',
    paddingTop: 35,
  },
  encabezado: {
    paddingHorizontal: 20,
    paddingBottom: 15,
    borderBottomWidth: 1,
    borderBottomColor: '#d9e2ec',
  },
  titulo: {
    fontSize: 22,
    fontWeight: 'bold',
    color: '#102a43',
  },
  subtitulo: {
    fontSize: 12,
    color: '#627d98',
    marginTop: 4,
  },
  lista: {
    padding: 16,
  },
  card: {
    backgroundColor: '#ffffff',
    borderRadius: 12,
    padding: 16,
    marginBottom: 12,
    elevation: 3,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.08,
    shadowRadius: 4,
  },
  cardHeader: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    marginBottom: 8,
  },
  idTexto: {
    fontSize: 13,
    fontWeight: 'bold',
```

```
    color: '#829ab1',
  },
  marcaTexto: {
    fontSize: 13,
    fontWeight: 'bold',
    color: '#2b6cb0',
    textTransform: 'uppercase',
  },
  descripcionTexto: {
    fontSize: 16,
    fontWeight: '600',
    color: '#243b53',
    marginBottom: 12,
  },
  cardFooter: {
    flexDirection: 'row',
    justifyContent: 'flex-end',
  },
  stockBadge: {
    fontSize: 12,
    fontWeight: 'bold',
    paddingVertical: 4,
    paddingHorizontal: 10,
    borderRadius: 6,
  },
  conStock: {
    backgroundColor: '#c6f6d5',
    color: '#22543d',
  },
  sinStock: {
    backgroundColor: '#fed7d7',
    color: '#742a2a',
  },
  centro: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 24,
  },
  textoEstado: {
    marginTop: 12,
    color: '#486581',
    fontSize: 15,
```

```
    textAlign: 'center',
  },
  textoError: {
    color: '#c53030',
    fontSize: 15,
    textAlign: 'center',
    marginBottom: 16,
  },
  botonReintentar: {
    backgroundColor: '#3182ce',
    paddingVertical: 10,
    paddingHorizontal: 20,
    borderRadius: 8,
  },
  textoBoton: {
    color: '#ffffff',
    fontWeight: 'bold',
  },
});
```

7. Paso 6: Ejecución y Prueba en el Teléfono

Una vez guardado el archivo App.js, iniciamos el servidor de desarrollo Metro Bundler desde la terminal de VS Code o PowerShell:

```
# Iniciar el servidor de desarrollo de Expo en red local
npx expo start

# Alternativa si la red Wi-Fi tiene aislamiento de clientes (Client Isolation)
npx expo start --tunnel
```

- Abre la aplicación Expo Go en tu teléfono Android (o la Cámara en iOS) y escanea el código QR que aparece en la terminal.
- Al abrirse la aplicación, se ejecutará el hook useEffect(), disparando fetch('https://progra3/api/base/').
- Deslizando el dedo hacia abajo sobre la lista (Pull to Refresh), la aplicación volverá a consultar la base de datos en tiempo real, reflejando cualquier cambio de stock o nuevos productos agregados en productos01.

> ⚠ **Consideración Técnica sobre Resolución de Nombres y Certificados SSL**
>
> Dado que el teléfono consulta https://progra3/api/base/, el nombre de dominio 'progra3' debe poder ser resuelto por el servidor DNS de la red Wi-Fi a la que esté conectado el celular, y el certificado HTTPS debe ser válido (por

> ejemplo, emitido por Let's Encrypt o una CA reconocida). Si 'progra3' es un dominio local con certificado autofirmado, React Native en Android/iOS puede rechazar la conexión SSL; en ese caso, durante las pruebas de laboratorio se recomienda utilizar protocolo HTTP (http://progra3/api/base/) o directamente la dirección IP del servidor.

Análisis del Código React Native (App.js)

Desglose de Estados, Ciclo de Vida, Promesas, FlatList y Guía de Actividades

El objetivo de esta parte es analizar cada bloque del archivo App.js para comprender cómo funciona React Native por dentro: desde las importaciones y el manejo de la memoria del componente (Hooks), hasta el consumo de APIs con Promesas y el renderizado eficiente de listas.

1. Bloque de Importaciones y Configuración Inicial

Todo archivo en React Native comienza trayendo a memoria los módulos y componentes necesarios mediante la sintaxis de desestructuración de JavaScript moderno (ES6):

```
import React, { useState, useEffect } from 'react';
import {
  StyleSheet,
  Text,
  View,
  FlatList,
  ActivityIndicator,
  SafeAreaView,
  StatusBar,
  TouchableOpacity
} from 'react-native';

const API_URL = 'https://progra3/api/base/';
```

1.1. ¿Qué traemos desde 'react'?

- **React:** Es el motor base que permite interpretar la sintaxis JSX (las etiquetas visuales dentro de JavaScript).
- **useState:** Es el Hook de Estado. Permite crear variables reactivas que conservan su valor entre renderizados y que, al actualizarse, redibujan la pantalla automáticamente.
- **useEffect:** Es el Hook de Efectos Secundarios (Ciclo de Vida). Permite ejecutar código automáticamente en momentos clave, como el instante exacto en que la pantalla se abre por primera vez.

1.2. ¿Qué traemos desde 'react-native'?

En el teléfono no existe un navegador web ni etiquetas HTML (como , <p> o <button>). En su lugar, importamos componentes nativos de Android e iOS:

Componente / Módulo	Equivalente Web (Aprox.)	Función en nuestra App de Productos
`SafeAreaView`	<main> con márgenes	Evita que el contenido quede tapado por la cámara frontal (notch) o la barra del sistema.
`View`		Contenedor básico que agrupa otros elementos usando Flexbox (en columna por defecto).
`Text`	<p> / / <h1>	Único componente capaz de mostrar texto en pantalla. Todo texto debe ir dentro de <Text>.
`FlatList`	+ .map()	Lista de alto rendimiento para mostrar los registros provenientes de MySQL.
`ActivityIndicator`	Spinner CSS	Rueda de carga animada nativa que se muestra mientras esperamos la respuesta del servidor.
`TouchableOpacity`	<button>	Área táctil que reduce levemente su opacidad al presionarla (usada en el botón Reintentar).
`StatusBar`	N/A (Sistema)	Controla el color de los íconos superiores del teléfono (batería, hora, señal Wi-Fi).
`StyleSheet`	Archivo .css	Utilidad que crea y valida el objeto de estilos visuales de la aplicación.

2. El Estado del Componente (useState): La Memoria de la Pantalla

Dentro de export default function App(), lo primero que declaramos son los tres estados que gobiernan nuestra pantalla mediante desestructuración de arreglos:

```
const [productos, setProductos] = useState([]);
const [cargando, setCargando] = useState(true);
const [error, setError] = useState(null);
```

¿Por qué necesitamos exactamente estos tres estados para trabajar con una base de datos?

- **1. productos y setProductos:** Almacena el arreglo (Array) de objetos JSON que nos devuelve la tabla productos01. Arranca como un arreglo vacío [] porque al milisegundo cero todavía no llegó la respuesta de Internet.
- **2. cargando y setCargando:** Es una bandera booleana (true/false). Arranca en true para que la pantalla muestre el indicador de carga (ActivityIndicator) mientras viaja la petición HTTP.
- **3. error y setError:** Almacena un mensaje de texto si algo falla (por ejemplo, si el celular no tiene Wi-Fi o el servidor está caído). Arranca en null (sin errores).

> 🧠 **Concepto Clave: ¿Por qué no usar una variable común (let productos = [])?**
>
> Si guardáramos los datos de MySQL en una variable común con 'let', cuando el fetch() terminara de descargar los productos y los asignara a la variable, la pantalla del teléfono NO se enteraría ni se actualizaría. Al invocar setProductos(json.data), no solo guardamos el dato, sino que le ordenamos a React Native que vuelva a dibujar (re-renderizar) la interfaz con los nuevos productos.

3. Asincronía y Promesas: La función consultarProductos()

La comunicación con el servidor https://progra3/api/base/ no es instantánea: los paquetes deben viajar por la red, el servidor debe consultar MySQL y devolver el JSON. Para no congelar la pantalla del teléfono mientras esperamos, usamos fetch(), que retorna una Promesa:

```
const consultarProductos = () => {
  setCargando(true);
  setError(null);

  fetch(API_URL)
    .then(response => {
      if (!response.ok) {
        throw new Error(`Error HTTP: ${response.status}`);
      }
      return response.json();
    })
    .then(json => {
      if (json.status === 'success') {
        setProductos(json.data);
      } else if (Array.isArray(json)) {
        setProductos(json);
      } else {
```

```
        throw new Error(json.message || 'Formato no reconocido');
      }
    })
    .catch(err => {
      console.error('Error al conectar con la base de datos:', err);
      setError('No se pudo obtener la información desde https://progra3/api/base/');
    })
    .finally(() => {
      setCargando(false);
    });
};
```

Paso a paso de la cadena de Promesas:

- **1. Preparación:** Antes de disparar la consulta, ponemos cargando en true y limpiamos cualquier error previo (null). Esto es fundamental cuando el usuario presiona el botón 'Reintentar'.
- **2. Primer .then(response => ...):** Recibe los encabezados de la respuesta HTTP. Verificamos response.ok (que es true si el código HTTP está entre 200 y 299). Si hubo un error 404 o 500, lanzamos una excepción con throw new Error() que salta directo al .catch(). Si todo está bien, ejecutamos return response.json(), que lee el flujo de datos y devuelve una nueva Promesa con el texto convertido en objeto JavaScript.
- **3. Segundo .then(json => ...):** Recibe los datos ya transformados en objeto JavaScript (json). Extrae el arreglo de filas de la tabla productos01 y llama a setProductos() para guardarlos en el estado.
- **4. Bloque .catch(err => ...):** Actúa como red de contención. Si el servidor no responde, falla el DNS o se lanzó un error en los .then() anteriores, captura el fallo y guarda un mensaje amigable mediante setError(...).
- **5. Bloque .finally(() => ...):** Se ejecuta siempre al terminar la petición (haya salido bien o haya dado error). Su única misión es apagar el indicador de carga llamando a setCargando(false).

4. El Ciclo de Vida con useEffect()

Declarar la función consultarProductos() no la ejecuta por sí sola. Para que la consulta a la base de datos se dispare automáticamente apenas el usuario abre la aplicación, utilizamos el Hook useEffect:

```
useEffect(() => {
  consultarProductos();
}, []); // <-- ¡Atención a este arreglo vacío!
```

- **El primer parámetro (() => { ... }):** Es la función que React ejecutará después de mostrar la pantalla por primera vez.
- **El segundo parámetro ([]):** Se denomina 'arreglo de dependencias'. Al dejarlo vacío [], le indicamos a React Native que ejecute consultarProductos() UNA SOLA VEZ al montarse el componente. Si

omitiéramos ese [], cada vez que llegaran los productos y se actualizara el estado, se volvería a disparar el fetch() creando un bucle infinito de peticiones al servidor.

5. Renderizado de cada Fila y Estilos Dinámicos (renderProducto)

La función renderProducto define cómo se debe dibujar visualmente cada registro individual que llega desde la tabla productos01:

```
const renderProducto = ({ item }) => {
  const sinStock = Number(item.stock) === 0;

  return (
    <View style={styles.card}>
      <View style={styles.cardHeader}>
        <Text style={styles.idTexto}>ID #{item.id}</Text>
        <Text style={styles.marcaTexto}>{item.marca}</Text>
      </View>

      <Text style={styles.descripcionTexto}>{item.descripcion}</Text>

      <View style={styles.cardFooter}>
        <Text style={[
          styles.stockBadge,
          sinStock ? styles.sinStock : styles.conStock
        ]}>
          {sinStock ? 'SIN STOCK' : `Stock: ${item.stock} unidades`}
        </Text>
      </View>
    </View>
  );
};
```

- **Desestructuración ({ item }):** FlatList le pasa a esta función un objeto que contiene cada fila dentro de la propiedad 'item'. Representa un registro de MySQL, por ejemplo: { id: 1, descripcion: 'Monitor LED...', marca: 'Samsung', stock: 15 }.
- **Cálculo de estado (sinStock):** Compara si el stock del producto es igual a cero, guardando true o false en la constante sinStock.
- **Arreglo de estilos condicionales:** En React Native, la propiedad style acepta un arreglo de estilos [estiloBase, estiloCondicional]. Siempre aplica styles.stockBadge (bordes redondeados y relleno), pero usa el operador ternario (?) para elegir el color: si sinStock es true aplica fondo rojo claro (styles.sinStock); si es false, aplica fondo verde claro (styles.conStock).

6. Renderizado Condicional de la Pantalla y el Componente FlatList

En el bloque return principal del componente, utilizamos operadores ternarios encadenados (condicion ? vistaA : vistaB) para decidir cuál de las tres vistas posibles mostrar según el estado actual:

```
{cargando ? (
  /* VISTA 1: Cargando... */
  <View style={styles.centro}>
    <ActivityIndicator size="large" color="#2b6cb0" />
    <Text style={styles.textoEstado}>Consultando base de datos...</Text>
  </View>
) : error ? (
  /* VISTA 2: Pantalla de Error con botón Reintentar */
  <View style={styles.centro}>
    <Text style={styles.textoError}>{error}</Text>
    <TouchableOpacity style={styles.botonReintentar} onPress={consultarProductos}>
      <Text style={styles.textoBoton}>Reintentar consulta</Text>
    </TouchableOpacity>
  </View>
) : (
  /* VISTA 3: Lista de Productos (FlatList) */
  <FlatList
    data={productos}
    keyExtractor={(item) => item.id.toString()}
    renderItem={renderProducto}
    contentContainerStyle={styles.lista}
    onRefresh={consultarProductos}
    refreshing={cargando}
  />
)}
```

Las 5 propiedades esenciales de <FlatList>:

- **1. data={productos}:** Recibe el arreglo de productos almacenado en nuestro estado (productos).
- **2. keyExtractor={(item) => item.id.toString()}:** React necesita un identificador único en formato texto (String) para cada fila de la lista, de modo que si cambia un solo producto sepa exactamente qué tarjeta redibujar. Aquí usamos la clave primaria 'id' de nuestra tabla MySQL convertida a texto con .toString().
- **3. renderItem={renderProducto}:** Le indica qué función debe invocar para dibujar cada elemento del arreglo.
- **4. onRefresh y refreshing:** Habilitan de forma nativa el gesto 'Pull to Refresh' (deslizar el dedo hacia abajo en la lista para volver a consultar la base de datos en vivo).

7. Cuestionario de Repaso y Comprensión (Para Resolver)

Responde las siguientes preguntas conceptuales analizando el código estudiado en clase:

Pregunta 1 — Seguridad y Arquitectura:

¿Por qué nuestra aplicación móvil en React Native utiliza fetch() para comunicarse con https://progra3/api/base/ en lugar de abrir una conexión directa al puerto 3306 del servidor MySQL desde el teléfono?

Pregunta 2 — El rol de useState:

¿Qué ocurriría en la pantalla del teléfono si en lugar de usar const [productos, setProductos] = useState([]) declaráramos una variable global let productos = [] y dentro del .then() hiciéramos productos = json.data? ¿Se verían los productos en pantalla? Justifica tu respuesta.

Pregunta 3 — El arreglo de dependencias de useEffect:

En la línea useEffect(() => { consultarProductos(); }, []), ¿qué sucedería con el tráfico de red y el servidor si eliminámaramos el segundo parámetro (los corchetes vacíos [])?

Pregunta 4 — Cadena de Promesas:

En la función consultarProductos(), ¿cuál es la diferencia entre el código que se coloca dentro del bloque .catch() y el código que se coloca dentro del bloque .finally()? ¿Por qué setCargando(false) se ubica en .finally()?

Pregunta 5 — Identificadores en Listas:

¿Para qué sirve la propiedad keyExtractor de FlatList y por qué es necesario aplicar .toString() sobre el campo item.id que proviene de la tabla productos01?

8. Actividades Prácticas de Programación (Desafíos en Clase)

A partir del proyecto funcional appProductos01, realiza las siguientes modificaciones progresivas en el código de App.js:

Actividad 1 (Nivel Inicial) — Contador Total de Registros:

Modifica el encabezado de la aplicación (<View style={styles.encabezado}>) para que muestre dinámicamente cuántos productos se descargaron desde la base de datos. Ejemplo: 'Total en catálogo: 6 productos'. Pista: recuerda que el estado 'productos' es un Array y puedes consultar su propiedad .length dentro de llaves {}.

Actividad 2 (Nivel Inicial) — Botón de Actualización Manual:

Además del gesto de deslizar hacia abajo, agrega en el encabezado un botón visible (<TouchableOpacity>) con el

texto ' 🔄 Actualizar' que al presionarlo (onPress) vuelva a ejecutar la función consultarProductos().

Actividad 3 (Nivel Intermedio) — Alerta de 'Últimas Unidades' (Stock Crítico):

Actualmente las etiquetas de stock se muestran en rojo si el stock es 0 y en verde si es mayor a 0. Modifica la función renderProducto y el objeto StyleSheet para incorporar un tercer estado visual: si el stock es mayor a 0 pero menor o igual a 5 (item.stock > 0 && item.stock <= 5), la etiqueta debe mostrarse en color amarillo/naranja con la leyenda '¡Últimas X unidades!'.

Actividad 4 (Nivel Intermedio) — Buscador en Tiempo Real por Marca o Descripción:

Importa el componente TextInput desde 'react-native' y crea un nuevo estado llamado const [busqueda, setBusqueda] = useState(''). Coloca una caja de texto arriba de la lista para que el usuario pueda escribir una marca (ej: 'Logitech') y filtra el arreglo que recibe la FlatList usando productos.filter(...) para mostrar solo los productos que coincidan.

Actividad 5 (Nivel Avanzado) — Incorporación del Campo 'precio':

1) En el servidor MySQL, agrega una columna 'precio DECIMAL(10,2) DEFAULT 0.00' a la tabla productos01 y asigna precios a los productos.
2) Actualiza la consulta SQL del endpoint si es necesario.
3) Modifica el componente renderProducto en App.js para mostrar el precio formateado con el signo '$' destacado en cada tarjeta.